Library Management System — Review III Report

Student: Suyash Singh
Register number: 24BYB0078
Course: BCSE302L / BCSE302P Database Systems Lab
Slot / **faculty:** G1 / Ramkumar T
Institution: Vellore Institute of Technology
Project type: Individual project

1. Introduction and problem statement

A library must know which books and copies it owns, who has borrowed them, when they are due, and which late fees remain unpaid. Manual registers can duplicate members, issue unavailable books, miss overdue returns, and miscalculate fines. This project keeps those records in a relational Oracle database and connects them to an interactive librarian dashboard.

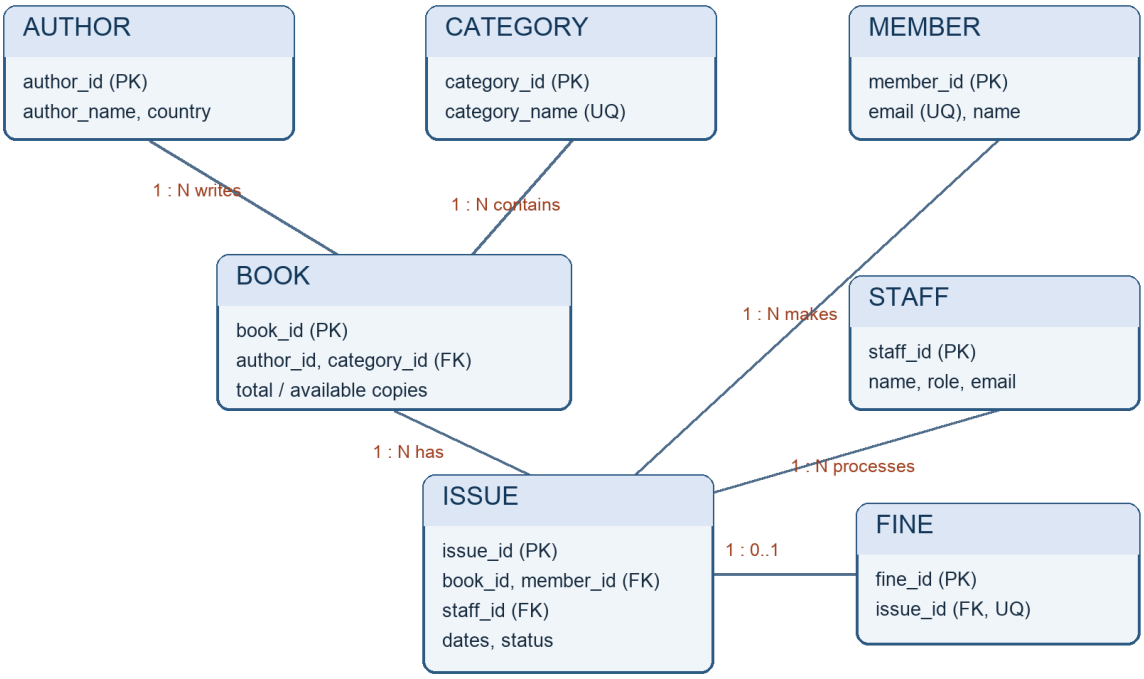
Review I defined the entities, relationships, and functional requirements. Review II created the tables, constraints, sample data, 15 queries, two procedures, two functions, an explicit cursor demonstration, and two triggers. Review III connects that database to a Flask API and browser interface. The browser interface extends the original Review I scope for the final integrated demonstration.

2. Functional requirements and completed system

Requirement	Implementation
Store books, authors, categories, members and staff	CRUD pages and Oracle tables `BOOK`, `AUTHOR`, `CATEGORY`, `MEMBER`, `STAFF`
Issue only available books	`issue_book` inserts an `ISSUE`; `trg_issue_insert` checks and decrements copies
Record due dates and returns	`ISSUE` dates/status; `return_book` procedure
Restore stock on return	`trg_issue_return` increments available copies on `Issued` to `Returned`
Calculate and track fines	`overdue_days`, `calculate_fine`, `FINE` table, payment-status update
Show activity and reports	Dashboard, issue history, overdue cursor page, 12 SQL-backed reports and CSV export
Demonstrate INSERT, UPDATE and DELETE	Member/book/author/category forms, fine payment, and deletion with FK protection

The application is a local librarian/admin demonstration. It has no member login, online payment, reservation or e-book feature.

3. ER diagram and relational schema



ER diagram showing AUTHOR and CATEGORY linked to BOOK, BOOK, MEMBER and STAFF linked to ISSUE, and the optional FINE for an ISSUE

`ISSUE` resolves the many-to-many relationship between books and members over time. `FINE.issue_id` is both a foreign key and unique, so an issue has at most one fine.

Relational schema:

```
AUTHOR(author_id PK, author_name, country)
CATEGORY(category_id PK, category_name UNIQUE)
BOOK(book_id PK, title, author_id FK→AUTHOR, category_id FK→CATEGORY,
     price, total_copies, available_copies)
MEMBER(member_id PK, name, email UNIQUE, phone, join_date, membership_type)
STAFF(staff_id PK, name, role, email UNIQUE)
ISSUE(issue_id PK, book_id FK→BOOK, member_id FK→MEMBER, staff_id FK→STAFF,
      issue_date, due_date, return_date, status)
FINE(fine_id PK, issue_id UNIQUE FK→ISSUE, amount, paid_status)
```

The exact Oracle DDL and constraints are in [sql/schema.sql](). They include primary and foreign keys, `NOT NULL`, `UNIQUE`, `CHECK`, and `DEFAULT`. The sample data in [sql/sample_data.sql]() has at least ten rows in every table: 10 authors, 10 categories, 10 books, 10 members, 10 staff, 15 issues, and 10 fines.

4. Database implementation and connectivity

```
Browser (HTML/CSS/JavaScript)
  → fetch /api/* JSON
Flask routes (validation, transactions, serialization)
  → python-oracledb connection pool
Oracle tables + SQL queries + PL/SQL procedures/functions/triggers
  → JSON results and dashboard/report output
```

The connection parameters are read from `.env` by [db.py](). Flask routes in [app.py]() use bound SQL parameters. Writes commit after dependent database work succeeds and roll back on Oracle errors. The issue and return routes call the Review II procedures; the application never manually adjusts stock for those

operations.

For a fresh database, run `sql/schema.sql`, then `sql/sample_data.sql`, then `sql/procedures.sql`. The sample inserts historical issues directly, so the triggers are installed **after** the sample data to avoid changing its prepared copy counts. Existing Review II databases should be verified instead of recreated. Run `python scripts/verify_oracle.py` after configuring credentials. A passing `/api/status` must show `connected: true` for the final Oracle demonstration. Demo mode uses static data and does not prove database integration.

5. SQL queries and outputs

The 15 Review II SQL queries are in `sql/queries.sql`. They cover joins, availability filtering, history, grouping, `LEFT JOIN`, `NOT EXISTS`, subqueries, `HAVING`, overdue dates, fine totals, and staff activity. The integrated Reports page offers 12 related live queries through `/api/reports/<name>`.

The following results were observed from the local Oracle `LIBRARY` schema on 26 October 2026. Full row-by-row output for all 15 queries is in the live verification appendix, which is also included in the PDF report.

Query / report	Observed result
Dashboard titles / copies / available	10 titles, 43 total copies, 38 available copies
Active and overdue issues	5 active issues, IDs 11–15; all 5 are overdue
Fine summary	Paid: ₹55 from 4 fines; Unpaid: ₹110 from 6 fines
Books by category	Fiction: 3; Classic: 2; Fantasy, Biography, History, Science, Mystery: 1 each; Self-Help, Technology, Education: 0
Staff activity	Staff IDs 1–5 handled 2 issues each; IDs 6–10 handled 1 each
Borrow count per member	Members 2–6 have 2 issues each; members 1 and 7–10 have 1 each
Inactive members	None in the sample

The appendix also records actual INSERT, UPDATE, DELETE, issue, return, fine and cursor results from reversible Oracle transactions. Those verification transactions were rolled back, leaving the sample data unchanged. The live result can change with the Oracle server date and later UI transactions. The CSV button exports the rows currently returned by each report endpoint.

6. Procedures, functions, cursor and triggers

All Review II PL/SQL definitions are in `sql/procedures.sql`.

Object	Role in an integrated operation
`issue_book` procedure	Inserts a new borrowing transaction
`return_book` procedure	Locks the issue, records the return and inserts a positive fine
`overdue_days` function	Calculates nonnegative late calendar days
`calculate_fine` function	Multiplies late days by ₹5
`trg_issue_insert`	Rejects unavailable books and decrements stock after issue insert
`trg_issue_return`	Restores one copy when status changes from `Issued` to `Returned`
`overdue_cursor`	Review II explicit cursor prints each overdue issue in SQL Developer

The Database Logic page runs the same overdue selection through a PL/SQL `SYS_REFCURSOR` and fetches its rows into the UI. This is an actual cursor invocation; the Review II block remains available for a SQL Developer demonstration.

7. Five real-world transaction demonstrations

Use the detailed steps in [docs/demo-checklist.md](). Record the generated IDs during the live run, because `MAX(id)+1` may differ from the sample.

Transaction 1 — Register a member (INSERT)

User action: Members → Register Member. Enter a unique email, name, phone and membership type.
Database operation: `INSERT INTO MEMBER` with `join_date DEFAULT SYSDATE`; unique email and membership type constraints apply.
Result: A new member appears in the member list. Verify with `SELECT * FROM MEMBER WHERE email = '<email>';`.

Transaction 2 — Issue an available book (procedure + trigger)

User action: Issue Book → select member, book, staff and due date.
Database operation: Flask calls `issue_book`; it inserts `ISSUE`. `trg_issue_insert` validates available stock and decrements `BOOK.available_copies`.
Result: New `Issued` row; availability decreases by one. Verify the new `ISSUE` row and selected `BOOK.available_copies`.

Transaction 3 — Return an overdue book (procedure + functions + trigger)

User action: Return Book → select an overdue active issue → Return.
Database operation: `return_book` locks the issue, calls `calculate_fine` and `overdue_days`, updates its return date and status, and inserts a fine if positive. `trg_issue_return` restores one available copy.
Result: `ISSUE.status = 'Returned'`; one more available copy; one unpaid `FINE` with ₹5 per late calendar day. Verify all three tables using the issue ID.

Transaction 4 — Mark a fine as paid (UPDATE)

User action: Fine Management → Mark Paid.
Database operation: `UPDATE FINE SET paid_status = 'Paid' WHERE fine_id = :fine_id`.
Result: The fine moves from unpaid to paid totals. This records payment status; no payment gateway is involved.

Transaction 5 — Find and export overdue issues (cursor + report)

User action: Database Logic → Run Overdue Cursor Demo, then Reports → Overdue Issues → Export CSV.
Database operation: The PL/SQL reference cursor reads `ISSUE`, `MEMBER` and `BOOK` rows with `status = 'Issued' AND TRUNC(due_date) < TRUNC(SYSDATE)`; the report runs a corresponding joined SQL query.
Result: A row per overdue issue, including member, book and due date. The CSV contains the displayed report rows.

Additional UPDATE and DELETE demonstrations are available through editing/deleting a member, book, author or category. Parent deletion is rejected when referenced child rows exist, demonstrating foreign key protection.

8. Verification, limits and conclusion

Verification on 26 October 2026: `venv/bin/python -m unittest discover -s tests -v` passed 39 tests, with two live tests skipped by default. `RUN_LIVE_ORACLE_TESTS=true venv/bin/python -m unittest tests.test_app.LiveOracleTests -v` passed both live tests. `venv/bin/python scripts/verify_oracle.py` confirmed all seven tables and six required stored objects. The [live verification appendix]() records outputs

from all 15 queries and reversible transactions using real Oracle procedures, functions, cursor and triggers.

The system integrates the seven-table database with the Review II SQL and PL/SQL logic and presents five meaningful workflows through the UI. The main limits are local-only use without authentication, nonpersistent demo-mode writes, and `MAX(id)+1` ID allocation under concurrent requests. For the final viva, open the website with Oracle connected and demonstrate the five workflows using the checklist.

Live Oracle Verification — 26 October 2026

This is an observed snapshot from the local `LIBRARY` schema in `FREEPDB1`. Queries come from `sql/queries.sql`. Data can change after later UI transactions.

Oracle server time: 2026-10-26 11:30:01

Table row counts

Table	Rows
AUTHOR	10
CATEGORY	10
BOOK	10
MEMBER	10
STAFF	10
ISSUE	15
FINE	10

Review II query outputs

1. Books with author and category

Rows returned: 10

TITLE	AUTHOR_NAME	CATEGORY_NAME
Malgudi Days	R. K. Narayan	Fiction
The Alchemist	Paulo Coelho	Fiction
Five Point Someone	Chetan Bhagat	Fiction
1984	George Orwell	Classic
Pride and Prejudice	Jane Austen	Classic
Harry Potter	J. K. Rowling	Fantasy
A Brief History of Time	Stephen Hawking	Science
Sapiens	Yuval Noah Harari	History
Murder on the Orient Express	Agatha Christie	Mystery
Wings of Fire	A. P. J. Abdul Kalam	Biography

2. Available books

Rows returned: 10

TITLE	AVAILABLE_COPIES
Malgudi Days	4

1984	3
Harry Potter	5
The Alchemist	4
Wings of Fire	3
Pride and Prejudice	3
Sapiens	4
Five Point Someone	5
A Brief History of Time	3
Murder on the Orient Express	4

3. Members who borrowed books

Rows returned: 10

NAME
Aditi Sharma
Rahul Verma
Neha Gupta
Arjun Mehta
Isha Nair
Karan Das
Pooja Rao
Dev Malhotra
Riya Kapoor
Manav Jain

4. Issue history

Rows returned: 15

ISSUE_ID	NAME	TITLE	ISSUE_DATE	DUE_DATE	RETURN_DATE	STATUS
1	Aditi Sharma	Malgudi Days	2026-08-01	2026-08-08	2026-08-11	Returned
11	Rahul Verma	Malgudi Days	2026-09-01	2026-09-15	—	Issued
2	Rahul Verma	1984	2026-08-02	2026-08-09	2026-08-12	Returned
12	Neha Gupta	1984	2026-09-02	2026-09-16	—	Issued
3	Neha Gupta	Harry Potter	2026-08-03	2026-08-10	2026-08-14	Returned
13	Arjun Mehta	Harry Potter	2026-09-03	2026-09-17	—	Issued
4	Arjun Mehta	The Alchemist	2026-08-04	2026-08-11	2026-08-13	Returned
14	Isha Nair	The Alchemist	2026-09-04	2026-09-18	—	Issued
5	Isha Nair	Wings of Fire	2026-08-05	2026-08-12	2026-08-15	Returned
15	Karan Das	Wings of Fire	2026-09-05	2026-09-19	—	Issued
6	Karan Das	Pride and Prejudice	2026-08-06	2026-08-13	2026-08-16	Returned
7	Pooja Rao	Sapiens	2026-08-07	2026-08-14	2026-08-18	Returned
8	Dev Malhotra	Five Point Someone	2026-08-08	2026-08-15	2026-08-18	Returned
9	Riya Kapoor	A Brief History of Time	2026-08-09	2026-08-16	2026-08-20	Returned

10	Manav Jain	Murder on the Orient Express	2026-08-10	2026-08-17	2026-08-21	Returned

5. Book titles per category

Rows returned: 10

CATEGORY_NAME	TOTAL
Fiction	3
Classic	2
Fantasy	1
Biography	1
History	1
Science	1
Mystery	1
Self-Help	0
Education	0
Technology	0

6. Borrow count per member

Rows returned: 10

NAME	BORROWED
Aditi Sharma	1
Rahul Verma	2
Neha Gupta	2
Arjun Mehta	2
Isha Nair	2
Karan Das	2
Pooja Rao	1
Dev Malhotra	1
Riya Kapoor	1
Manav Jain	1

7. Members with no borrowing history

Rows returned: 0

No rows in this snapshot.

8. Books above average price

Rows returned: 4

TITLE	PRICE
1984	399.0
Harry Potter	499.0
Sapiens	599.0
A Brief History of Time	450.0

9. Overdue active issues

Rows returned: 5

ISSUE_ID	NAME	TITLE	DUE_DATE
11	Rahul Verma	Malgudi Days	2026-09-15
12	Neha Gupta	1984	2026-09-16
13	Arjun Mehta	Harry Potter	2026-09-17
14	Isha Nair	The Alchemist	2026-09-18
15	Karan Das	Wings of Fire	2026-09-19

10. Total fines by payment status

Rows returned: 2

PAID_STATUS	TOTAL_FINE
Unpaid	110
Paid	55

11. Unpaid fines

Rows returned: 6

NAME	TITLE	AMOUNT
Aditi Sharma	Malgudi Days	15.0
Neha Gupta	Harry Potter	20.0
Isha Nair	Wings of Fire	15.0
Pooja Rao	Sapiens	20.0
Riya Kapoor	A Brief History of Time	20.0
Manav Jain	Murder on the Orient Express	20.0

12. Authors with multiple books

Rows returned: 0

No rows in this snapshot.

13. Most expensive book

Rows returned: 1

TITLE	PRICE
Sapiens	599.0

14. Issues handled per staff member

Rows returned: 10

NAME	HANDLED
Aarav Sharma	2
Diya Verma	2
Rohan Gupta	2
Meera Nair	2

Kabir Singh	2
Ananya Rao	1
Ishaan Das	1
Sara Khan	1
Vivaan Joshi	1
Priya Iyer	1

15. Books with copies currently issued

Rows returned: 5

TITLE	TOTAL_COPIES	AVAILABLE_COPIES
Malgudi Days	5	4
1984	4	3
Harry Potter	6	5
The Alchemist	5	4
Wings of Fire	4	3

Reversible integrated transaction demonstration

These operations ran against the real Oracle procedures, functions, triggers and tables in one transaction. The transaction was rolled back after verification, so the original sample data remains intact.

Step	Observed result
Register member (INSERT)	MEMBER 11: Review 3 Test Member, Student
Issue book through `issue_book` and `trg_issue_insert`	ISSUE 16 became Issued; BOOK 1 available copies 4 → 3
Overdue PL/SQL cursor	Included ISSUE 16 with due date 2026-10-25
Return through `return_book`, `calculate_fine`, `overdue_days`, `trg_issue_return`	ISSUE 16 became Returned; return date set; available copies 3 → 4; fine ₹5.0 Unpaid
Mark existing fine paid (UPDATE)	FINE 1 became Paid inside the transaction
Roll back test transaction	Test member and issue absent; FINE 1 remained Unpaid; BOOK 1 available copies remained 4

Additional DML check: inserted then deleted an unreferenced test MEMBER 11; DELETE affected 1 row and the row count was 0. The enclosing transaction was rolled back.